

Experiment 3: Information Theory

Aim

To calculate the information gain of all attributes using the Play Tennis dataset and identify the attribute with the highest information gain.

Tools Required

- Python
- Pandas
- Scikit-learn
- VS Code **Code**

```
import pandas as pd

from sklearn.preprocessing import LabelEncoder

from sklearn.tree import DecisionTreeClassifier

data = {

    "Outlook":["Sunny","Sunny","Overcast","Rain","Rain","Rain","Overcast","Sunny","Sunny","Rain","Sunny","Overcast","Overcast","Rain"],

    "Temperature":["Hot","Hot","Hot","Mild","Cool","Cool","Cool","Mild","Cool","Mild","Mild","Mild","Hot","Mild"],

    "Humidity":["High","High","High","High","Normal","Normal","Normal","High","Normal","Normal","Normal","High","Normal","High"],

    "Wind":["Weak","Strong","Weak","Weak","Weak","Strong","Strong","Weak","Weak","Weak","Weak","Strong","Strong","Weak","Strong"],

    "Play":["No","No","Yes","Yes","Yes","No","Yes","No","Yes","Yes","Yes","Yes","Yes","Yes","No"]

}

df = pd.DataFrame(data)

encoder = LabelEncoder()

X = df.drop("Play", axis=1).apply(encoder.fit_transform)

y = encoder.fit_transform(df["Play"])

model = DecisionTreeClassifier(criterion="entropy")

model.fit(X, y)

importance = model.feature_importances_

for feature, value in zip(X.columns, importance):

    print(feature, ":", value)

print("Highest Information Gain Attribute:", X.columns[importance.argmax()])
```

Output

```
E: > playtennis.py > ...
1 import pandas as pd
2 from sklearn.preprocessing import LabelEncoder
3 from sklearn.tree import DecisionTreeClassifier
4 data = {
5     'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
6     'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Cool', 'Mild', 'Mild', 'Mild', 'Hot', 'Mild'],
7     'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'Normal', 'Normal', 'Normal', 'High', 'Normal'],
8     'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
9     'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
10 }
11 df = pd.DataFrame(data)
12 encoder = LabelEncoder()
13 X = df.drop("Play", axis=1).apply(encoder.fit_transform)
14 y = encoder.fit_transform(df["Play"])
15 model = DecisionTreeClassifier(criterion="entropy")
16 model.fit(X, y)
17 importances = model.feature_importances_
18 print("Information Gain")

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[[0.07787826 0.92212174]]
Prediction: spam
PS C:\Users\chandravardhan reddy\AppData\Local\Programs\Microsoft VS Code> "C:\Users\chandravardhan reddy\AppData\Local\Python\pythoncore-3.14-64\python.exe"
e:/smsfraud.py
At line:1 char:84
+ ... dy\AppData\Local\Python\pythoncore-3.14-64\python.exe" e:/smsfraud.py
+ ~~~~~
Unexpected token 'e:/smsfraud.py' in expression or statement.
+ CategoryInfo          : ParserError: (:) [], ParentContainsErrorRecordException
+ FullyQualifiedErrorId : UnexpectedToken

PS C:\Users\chandravardhan reddy\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\chandravardhan reddy\AppData\Local\Python\pythoncore-3.14-64\python.exe"
" e:/playtennis.py
Information Gain
Outlook : 0.3626285821279446
Temperature : 0.15192946522259376
Humidity : 0.21123657918595443
Wind : 0.2742053734635072

Highest Information Gain Attribute: Outlook
```

Result

Information Theory was successfully implemented using the Play Tennis dataset, and **Outlook** was identified as the attribute with the highest information gain.